The F# Path to Relaxation

Language Design, Tools, Platforms, Community, Ecosystem, Exosystem, Openness, Cross-platform, …

Don Syme, F# Community Contributor

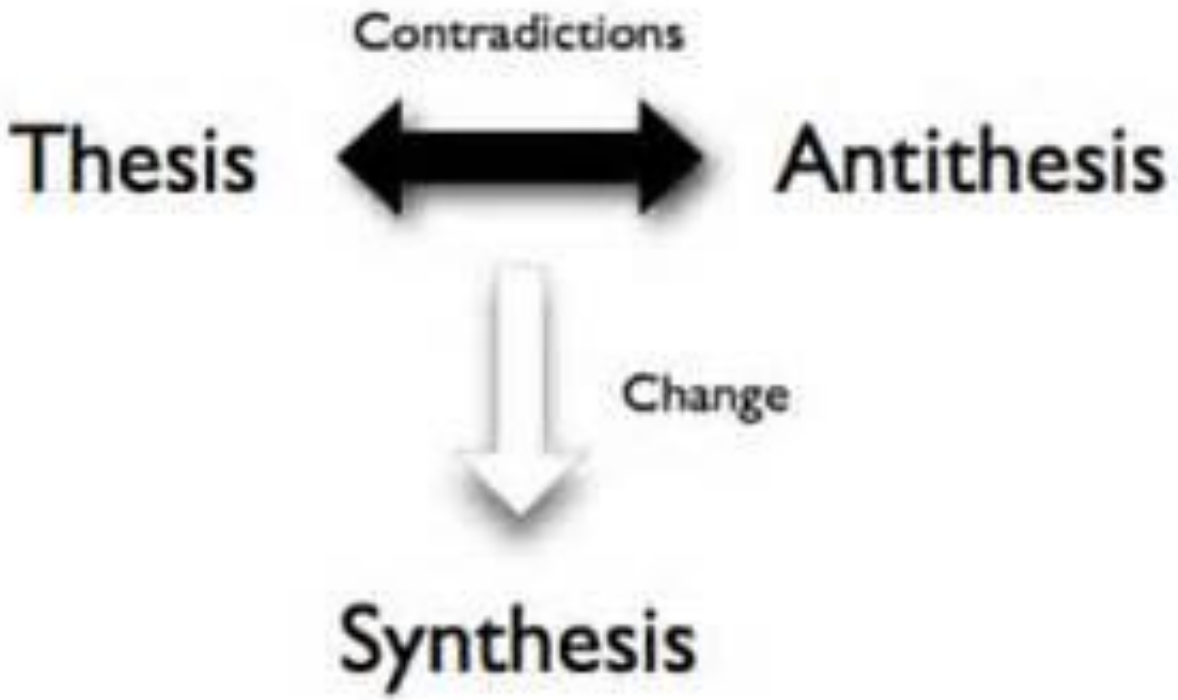
A Gentle Stroll Through the Land of Language Design and Delivery

A starting position….

The great disputes of computer science should be struggled with

They are chances to make a better, simpler, more relaxed world as much as create opposing camps

Not a new idea…



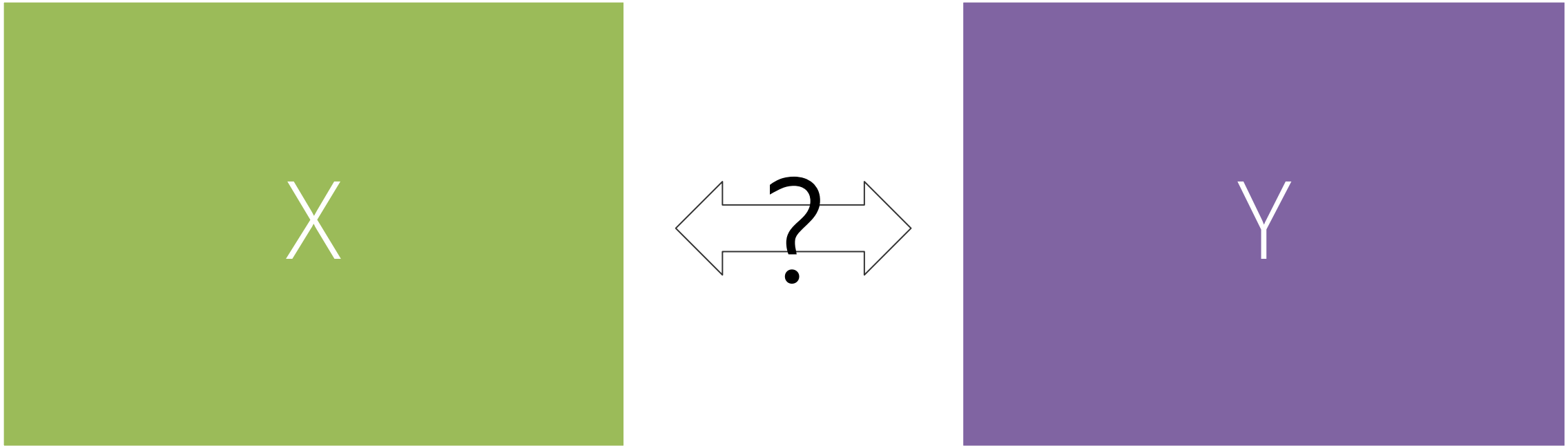
MUSTARD WATCHES :

AN INTEGRATED APPROACH TO TIME AND FOOD

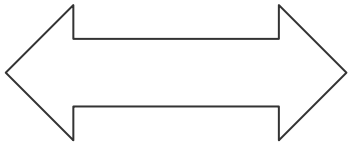
extended abstract

yann–joachim ringard
département de mathématiques
université libre Saint Augustin
21251 Quarnappes
ringard@brejnev.ustag.fr

The paper introduces the concept of mustard watch, *a common generalisation of the concepts of watch and of mustard pot. The main property of mustard watches is that they can deliver mustard in any desired quantity (theorem 1) and still display time with a precision of 30 seconds (theorem 2). But the real superiority of mustard watches over classical ones is expressed by our theorem 3 : a mustard watch with no mustard in it is at least as precise as an ordinary one.*


X
?
Y

Functional



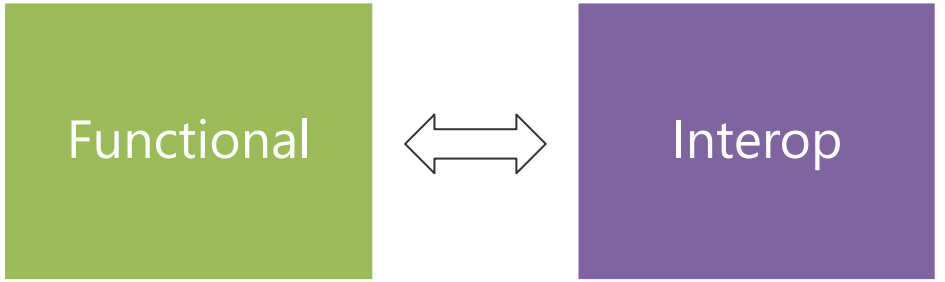
Interop

Then (2003)

Functional languages were **isolated**, **non-interoperable**, **using their own VMs**. Interop standards like C-calls, COM, CORBA, XML were a mess for language-integration.

The F# Approach (also Scala, Swift, …)

Embrace industry-standard runtime layers, and influence them. Keep exosystem at an appropriate distance. Design with end-to-end interop in mind. Compromise where needed.

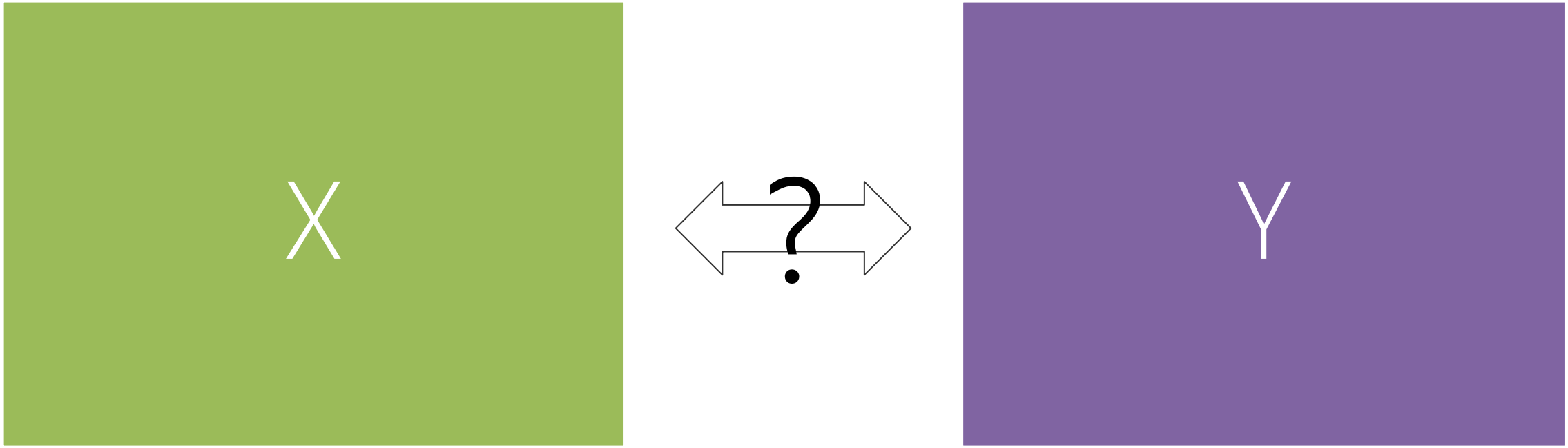


Relaxation Achieved, 200% ✓

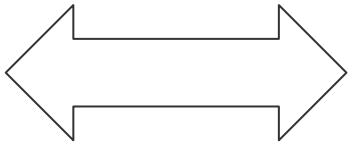
✓ Today's FP languages are immensely interoperable while staying true to FP principles.

✓ F# type providers raise interop to a completely new level

✗ Some tensions remain – not all functional techniques are easily implemented on industry standard VMs


X
?
Y

Enterprise



Openness

TL;DR – Where we are now

F# is open, cross-platform, neutral, independent

The F# language is accepting contributions

fsharp.org

Xamarin provide F# tools Android and iOS

The Visual F# Tools are Microsoft's professional tools for Windows and Azure

Debian and other packages for Linux

The F# Compiler Service powers many other tools

The F# community is very "self-empowered"

F# 4.0 now complete!

In one year, we've come a long way!

The F# Compiler – 300+ Pull Requests and rising

Where to Contribute?

See fsharp.github.io

The F# Language Design Process

fslang.uservoice.com

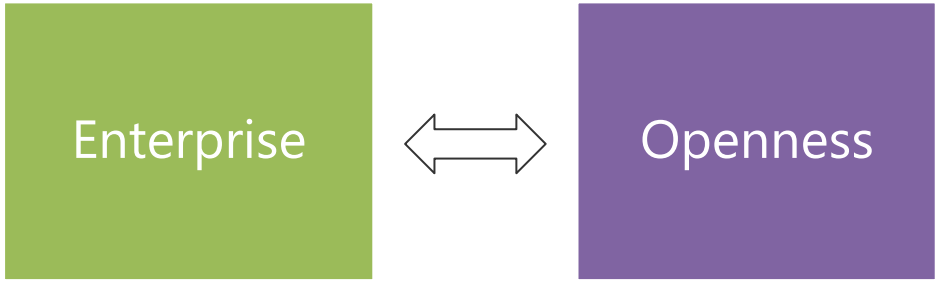
F# 4.0

- ✓ True shift to cross-platform open engineering
- ✓ Long laundry list of language items
- ✓ Normalized core library
- ✓ Type providers more powerful

- ✓ Better debugging, tooling, performance
- ✓ ~20% compiler perf improvement

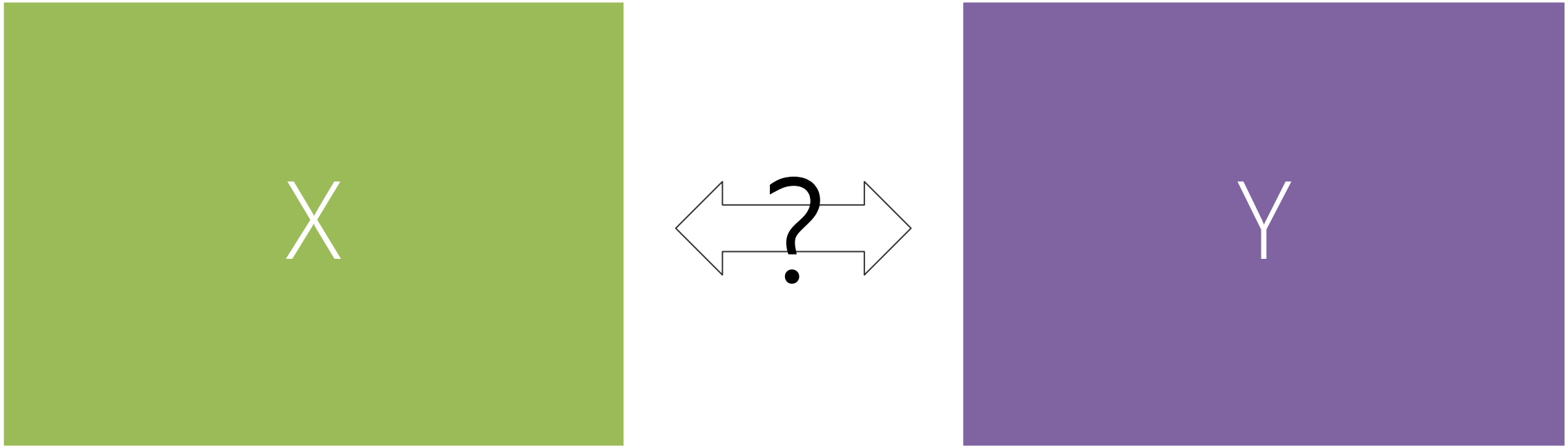
The F# Compiler Service

fsharp.github.io/FSharp.Compiler.Service

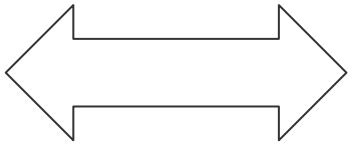


Relaxation Achieved ✓

Enterprise Quality + Openness +
Community + Tooling +
Ecosystem + Evolution Path
=
Goodness


X
?
Y

Functional



Objects

Then (2003)

In the 1990s functional languages were historically anti-OO

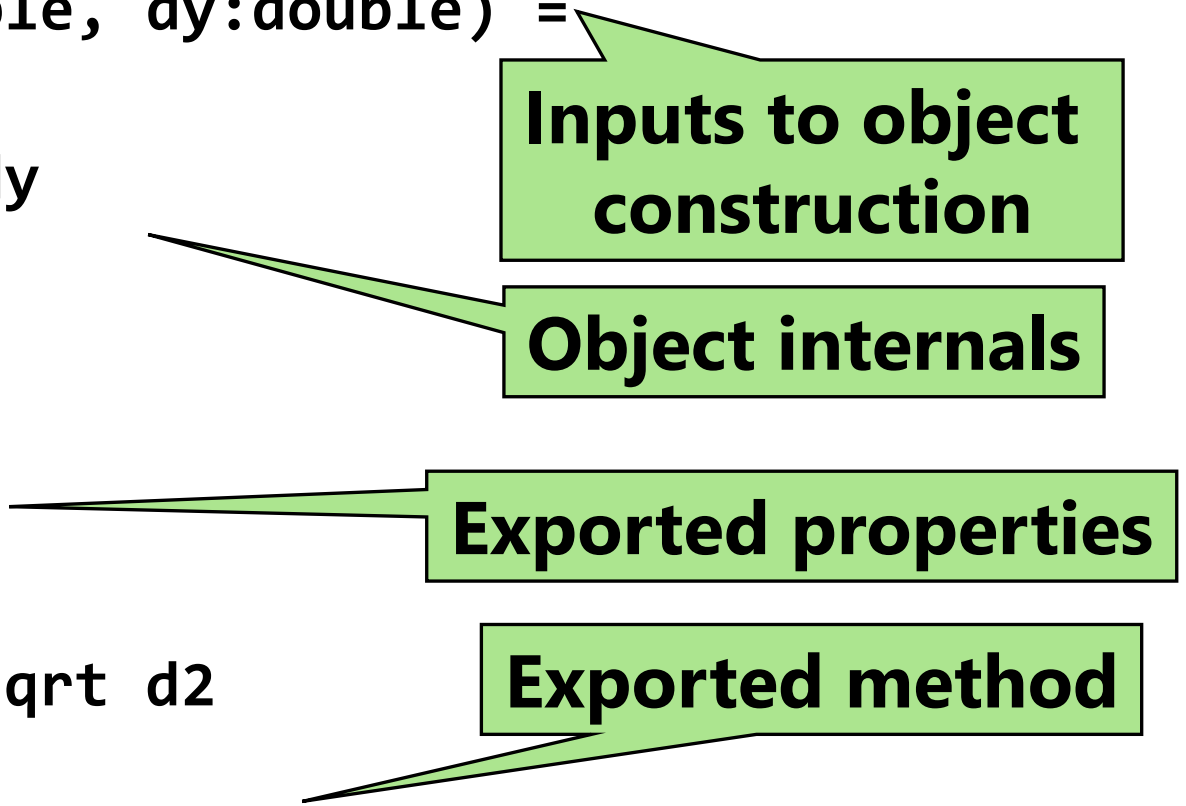
Even today any mention of objects gives some people an “urrgghhh…” reaction.

Modern Compiler Implementation with Standard ML - Chapter 14, Object Oriented Programming

object – to express a dislike or distaste for something

F# - Objects + Functional

```
type Vector2D (dx:double, dy:double) =

    let d2 = dx*dx+dy*dy

    member v.DX = dx

    member v.DY = dy

    member v.Length = sqrt d2

    member v.Scale(k) = Vector2D (dx*k, dy*k)
```



Objects

Constructed Class Types

```
type ObjectType(args) =
    let internalValue = expr
    let internalFunction args = expr
    let mutable internalState = expr
    member x.Prop1 = expr
    member x.Meth2 args = expr
```

Object Interface Types

```
type IObject =
  interface ISimpleObject
  abstract Prop1 : type
  abstract Meth2 : type -> type
```

Object Expressions

```
{ new IObject with
    member x.Prop1 = expr
    member x.Meth1 args = expr }

{ new Object() with
    member x.Prop1 = expr
    interface IObject with
      member x.Meth1 args = expr
    interface IWidget with
      member x.Meth1 args = expr }
```

The F# approach is to embrace objects, make them fit with the expression-oriented typed functional paradigm

especially for interop and software engineering purposes

but not embrace full "object-orientation"

Functional

Objects

	Expression-oriented	No null	Multiple Args = Tuples	Closure and Capture	First-class Values	Currying	HM Type Inference
void (unit)	✓	✓	✓	✓	✓	✓	✓
Object Types	✓	✓	✓	✓	✓		
Subtyping	✓	✓	✓	✓	✓	✓	
Dot-notation	✓	✓	✓	✓	✓	✓	¾
Inheritance	✓	✓				✓	
Method Overloding	✓	✓	✓	✓	¾	✗	¾

Relatively Graceful Degradation (some type annotations needed)

Relatively Graceful Degradation (some type annotations needed)

Some combinations outlawed for sanity

A functional-first approach makes a huge difference in practice

fsharp.org/testimonials

An analysis (Simon Cousins, Energy Sector)

350,000

lines of C# OO
by offshore team

The C# project took five years and peaked at ~8 devs. It never fully implemented all of the contracts.

The F# project took less than a year and peaked at three devs (only one had prior experience with F#). All of the contracts were fully implemented.

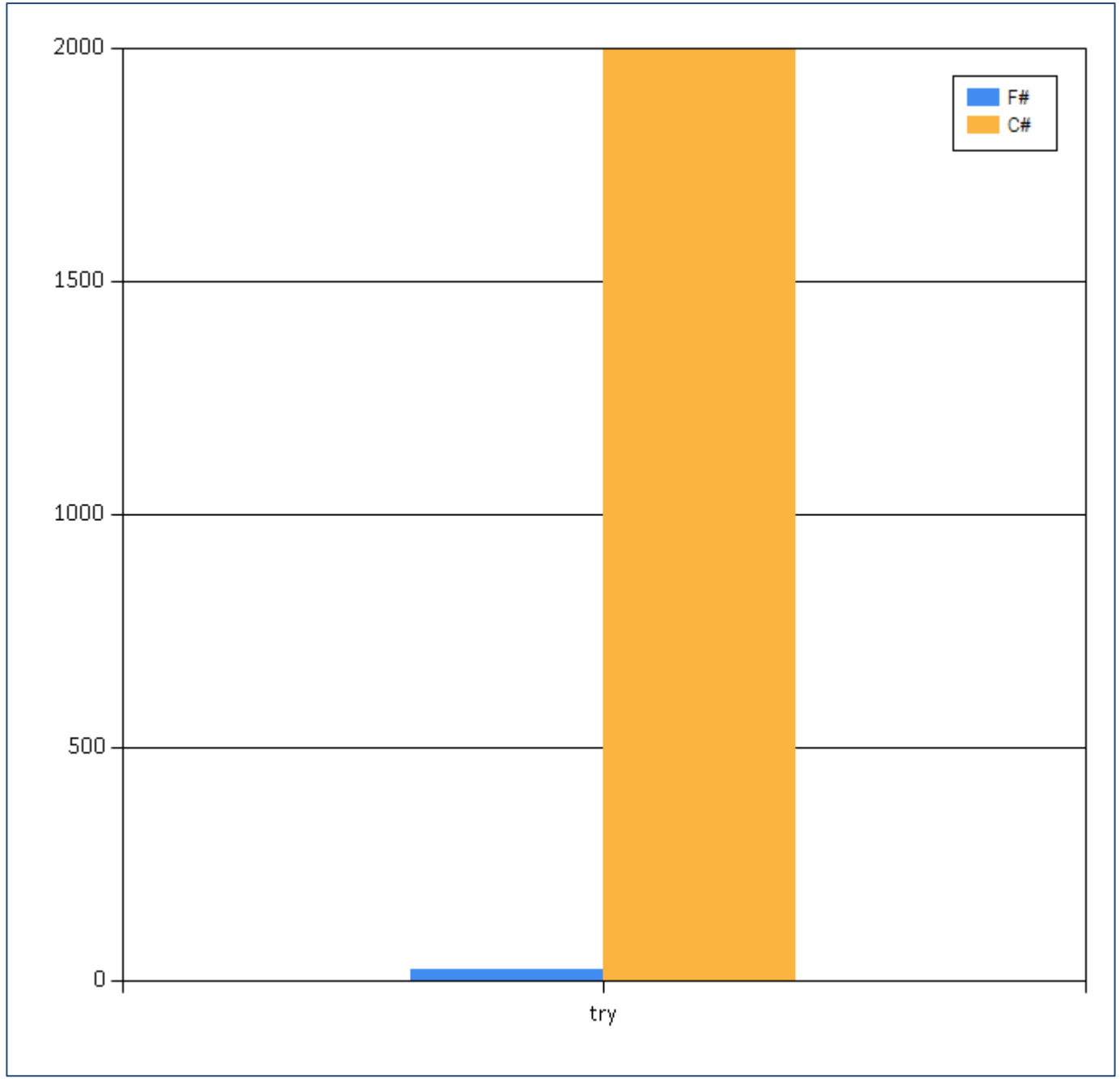
30,000

lines of robust F#, with parallel +more features

An application to evaluate the revenue due from Balancing Services contracts in the UK energy industry

http://simontcousins.azurewebsites.net/does-the-language-you-use-make-a-difference-revisited/

Implementation	C#	F#
Braces	56,929	643
Blanks	29,080	3,630
Null Checks	3,011	15
Comments	53,270	487
Useful Code	163,276	16,667
App Code	305,566	21,442
Test Code	42,864	9,359
Total Code	348,430	30,801


2000
1500
1000
500
0
F#
C#
try

Simon Cousins, Energy Sector

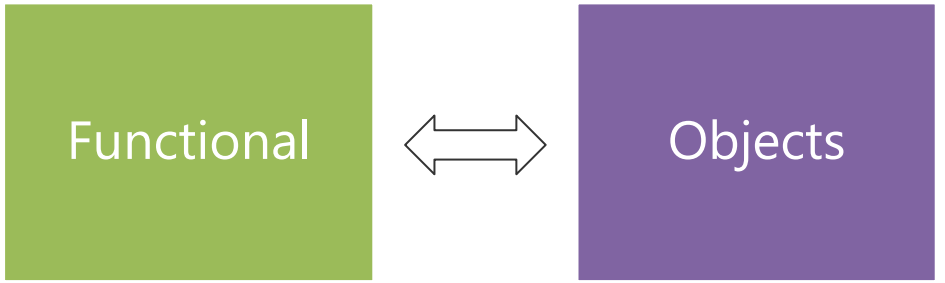
Zero

bugs in deployed system

"F# is the safe choice for this project,
any other choice is too risky"

An application to evaluate the revenue due from Balancing Services contracts in the UK energy industry

http://simontcousins.azurewebsites.net/does-the-language-you-use-make-a-difference-revisited/



Positive Synthesis Achieved ✓

some limitations remain… and there is one area in particular that's worth discussing….

Circularities and Modularity in the Wild

(Analysis by Scott Wlaschin and F# Community)

Mixed OO/Functional Programming Has Won

Lambdas in C#, Java, C++, …
Async monadic modality in C#, Javascript, PHP (Hack), …
Function types in C#, TypeScript, …
Generics in C#, Java, Visual Basic, …
…

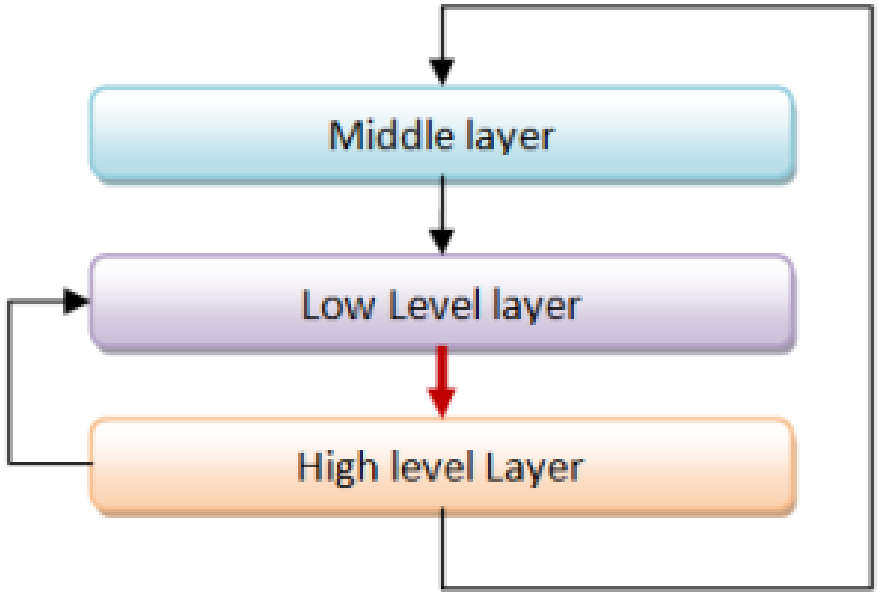
… except where it hasn't

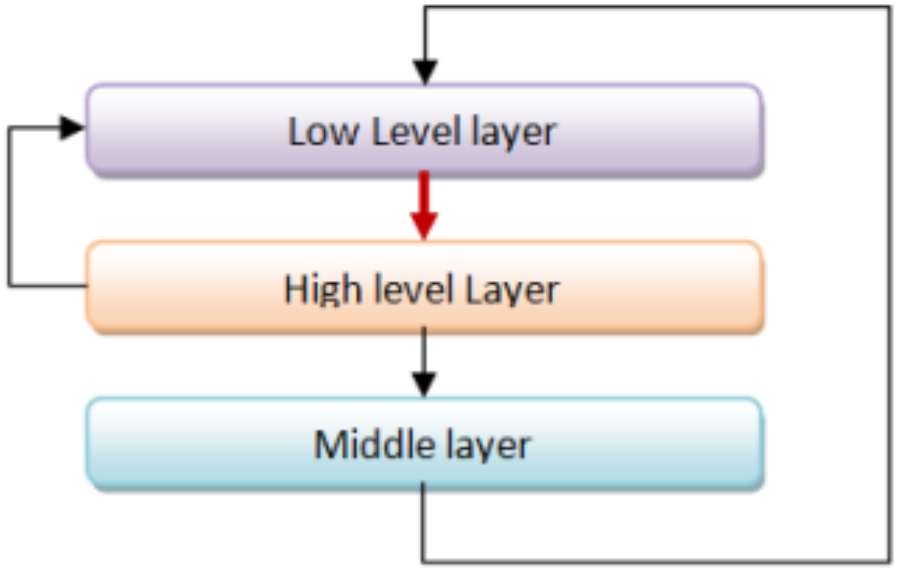
Inheritance everywhere!!!

Nulls everywhere!!!

Circularities everywhere!!!!

Unnecessary Circularities are Evil





To the functional programmer, it is “obvious” that our software methodology should help minimize and reduce cyclic dependencies in program structure.

The C# approach to circularity

All files in an “assembly” are mutually referential

Arbitrary circularity between “internal” items in an assembly

Mutually recursive “assemblies” are possible if you try hard

The F# approach to circularity

Like Haskell and OCaml, F# has a file ordering

F# encourages minimizing dependencies within a file

Parameterization the preferred technique

F# objects support limited forms of circularity when needed

Let's analyse some C# and F# projects

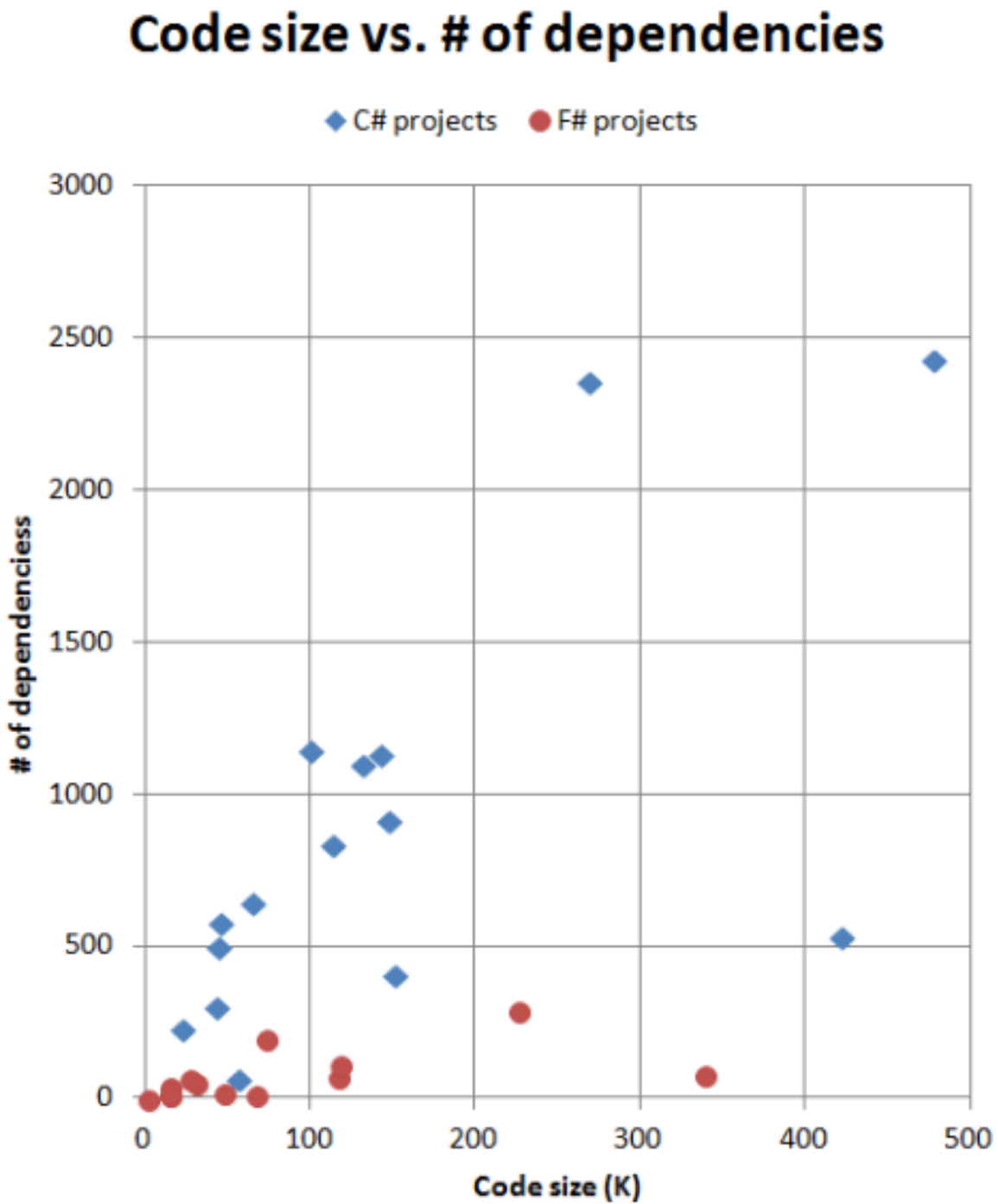
C# projects

- Mono.Cecil, which inspects programs and libraries in the ECMA CIL format.
- NUnit
- SignalR for real-time web functionality.
- NancyFx, a web framework
- YamlDotNet, for parsing and emitting YAML.
- SpecFlow, a BDD tool.
- Json.NET.
- Entity Framework.
- ELMAH, a logging framework for ASP.NET.
- NuGet itself.
- Moq, a mocking framework.
- NDepend, a code analysis tool.
- And, to show I'm being fair, a business application that I wrote in C#.

Let's analyse some C# and F# projects

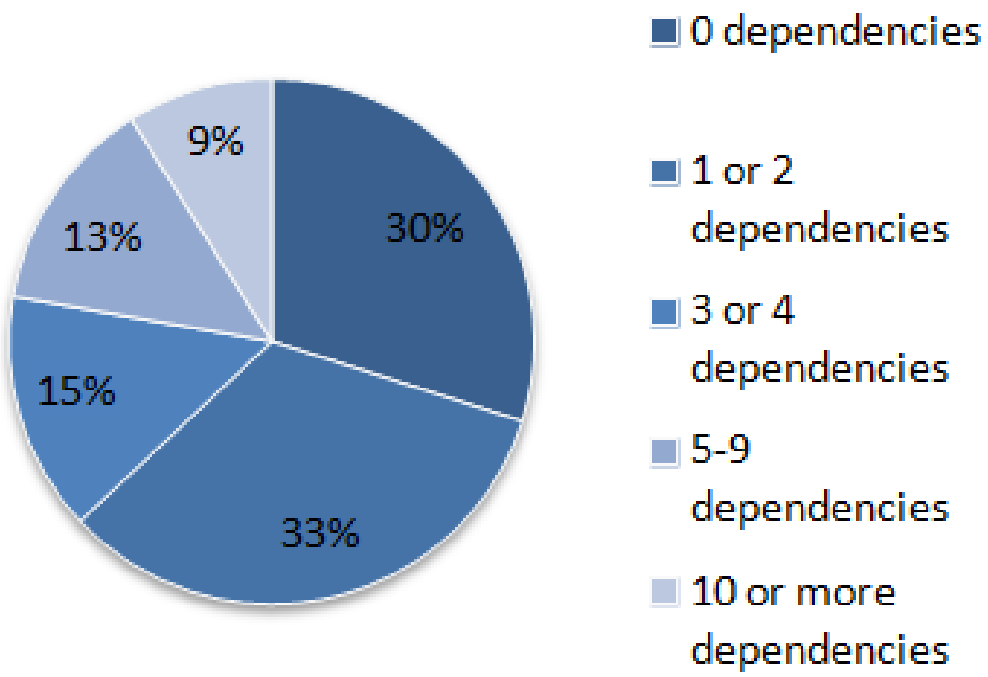
F# projects

- FSharp.Core, the core F# library.
- FSPowerPack.
- FsUnit, extensions for NUnit.
- Canopy, a wrapper around the Selenium test automation tool.
- FsSql, a nice little ADO.NET wrapper.
- WebSharper, the web framework.
- TickSpec, a BDD tool.
- FSharpx, an F# library.
- FParsec, a parser library.
- FsYaml, a YAML library built on FParsec.
- Storm, a tool for testing web services.
- Foq, a mocking framework.
- Another business application that I wrote, this time in F#.

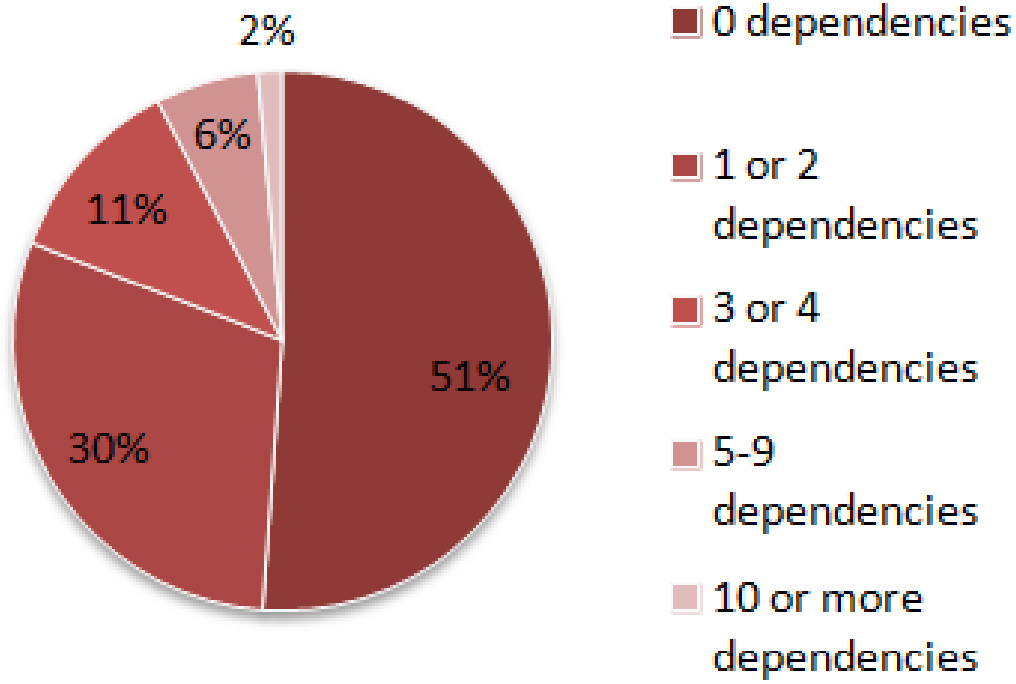


F# code has fewer top-level dependencies

C# projects
Top level types grouped by number of dependencies

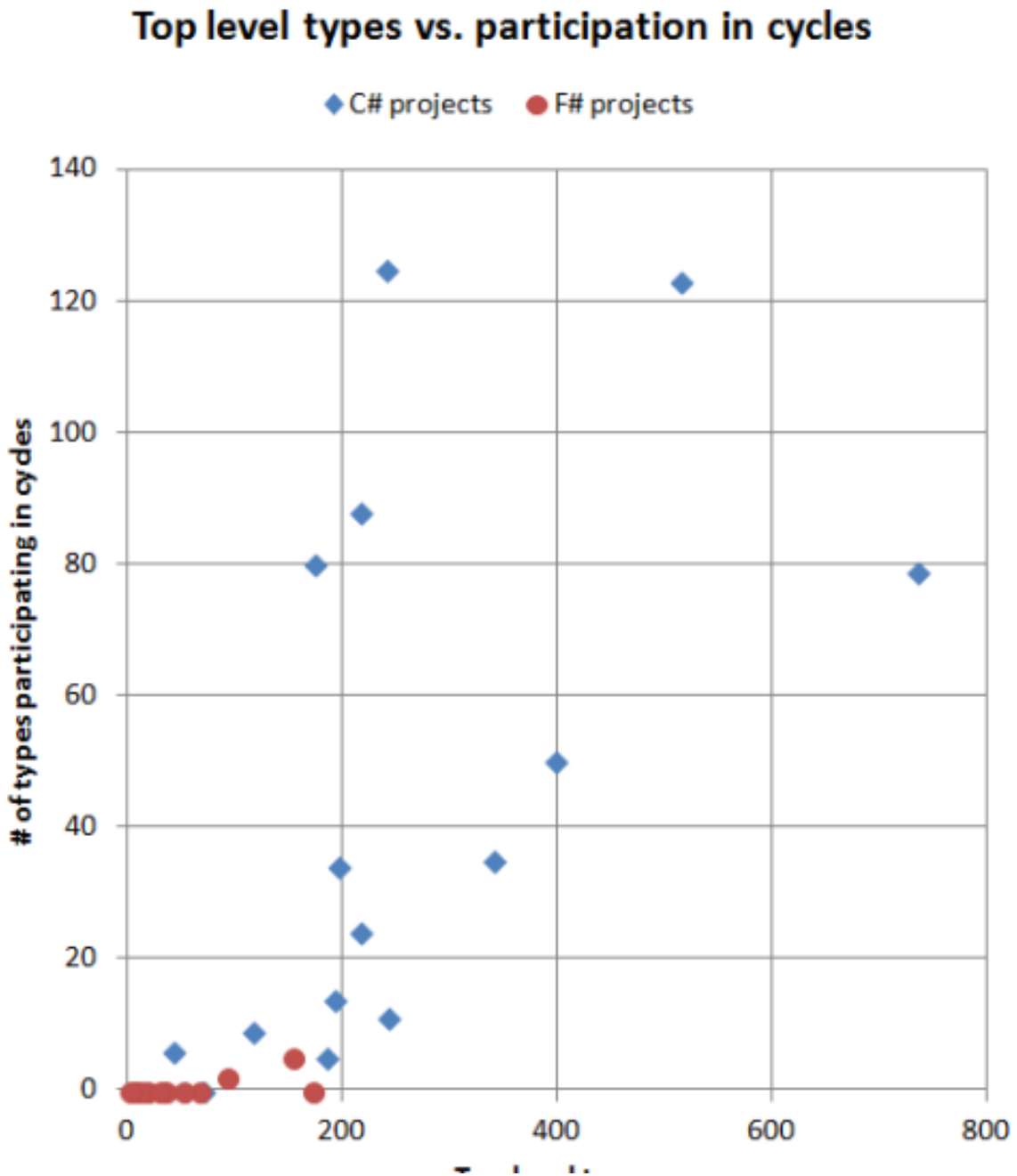


F# projects:
Top level types grouped by number of dependencies



Project	Top-level types	Cycle count	Partic.	Partic.%	Max comp. size	Cycle count (public)	Partic. (public)	Partic.% (public)	Max comp. size (public)
ef	514	14	123	24%	79	1	7	1%	7
jsonDotNet	215	3	88	41%	83	1	11	5%	11
nancy	339	6	35	10%	21	2	4	1%	2
cecil	240	2	125	52%	123	1	50	21%	50
nuget	216	4	24	11%	10	0	0	0%	1
signalR	192	3	14	7%	7	1	5	3%	5
nunit	173	2	80	46%	78	1	48	28%	48
specFlow	242	5	11	5%	3	1	2	1%	2
elmah	116	2	9	8%	5	1	2	2%	2
yamlDotNet	70	0	0	0%	1	0	0	0%	1
fparsecCS	41	3	6	15%	2	1	2	5%	2
moq	397	9	50	13%	15	0	0	0%	1
ndepend	734	12	79	11%	22	8	36	5%	7
ndependPlat	185	2	5	3%	3	0	0	0%	1
personalCS	195	11	34	17%	8	5	19	10%	7
TOTAL	3869		683	18%			186	5%	

Project	Top-level types	Cycle count	Partic.	Partic.%	Max comp. size	Cycle count (public)	Partic. (public)	Partic.% (public)	Max comp. size (public)
fsxCore	173	0	0	0%	1	0	0	0%	1
fsCore	154	2	5	3%	3	0	0	0%	1
fsPowerPack	93	1	2	2%	2	0	0	0%	1
storm	67	0	0	0%	1	0	0	0%	1
fParsec	8	0	0	0%	1	0	0	0%	1
websharper	52	0	0	0%	1	0	0	0%	0
tickSpec	34	0	0	0%	1	0	0	0%	1
websharperHtml	18	0	0	0%	1	0	0	0%	1
canopy	6	0	0	0%	1	0	0	0%	1
fsYaml	7	0	0	0%	1	0	0	0%	1
fsSql	13	0	0	0%	1	0	0	0%	1
fsUnit	2	0	0	0%	0	0	0	0%	0
foq	35	0	0	0%	1	0	0	0%	1
personalFS	30	0	0	0%	1	0	0	0%	1
TOTAL	692		7	1%			0	0%	



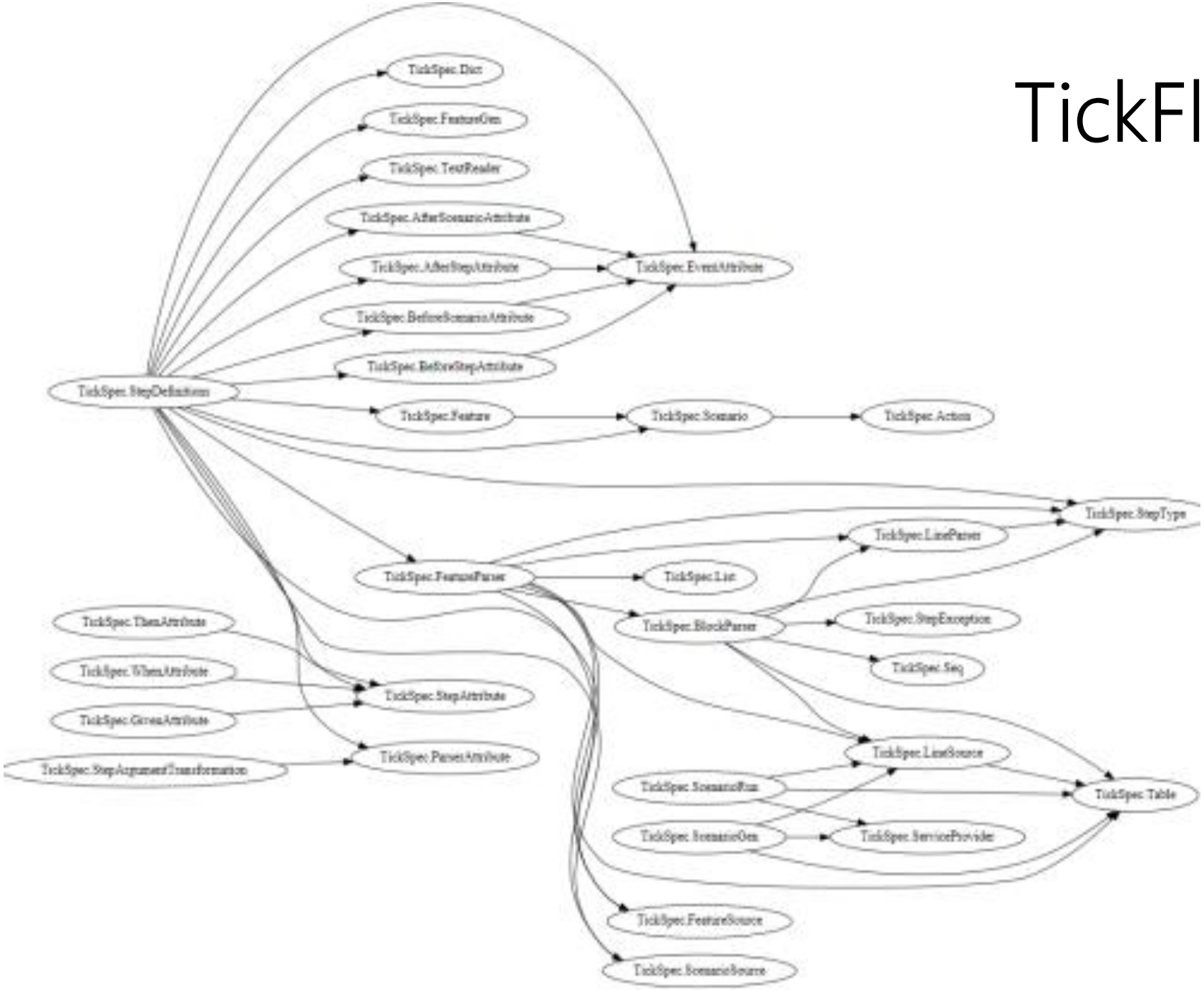
Why the difference between C# and F#?

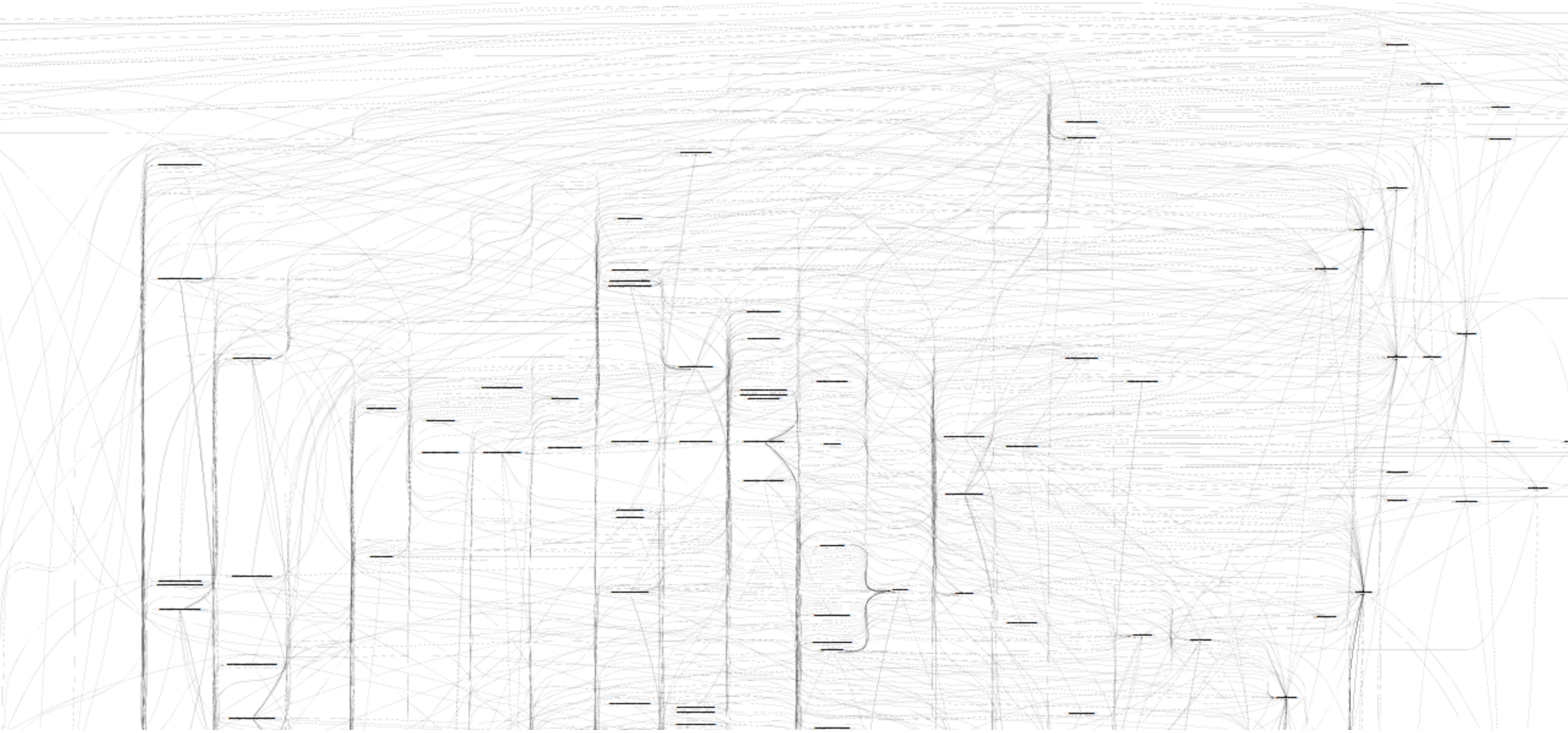
In C#, there is nothing stopping you from creating cycles. In fact, you have to make a special effort to avoid them.

In F#, you can't easily create cycles at all.

SpecFlow (C#)

TickFlow (F#)


TickSpec.Dirr
TickSpec.FeatureGen
TickSpec.TextReader
TickSpec.AfterScenarioAttribute
TickSpec.AfterStepAttribute
TickSpec.BeforeScenarioAttribute
TickSpec.BeforeStepAttribute
TickSpec.EventAttribute
TickSpec.StepDefinition
TickSpec.Feature
TickSpec.Scenario
TickSpec.Action
TickSpec.StepType
TickSpec.LineParser
TickSpec.FeatureParser
TickSpec.List
TickSpec.BlockParser
TickSpec.StepException
TickSpec.Seq
TickSpec.ThenAttribute
TickSpec.WhenAttribute
TickSpec.GivenAttribute
TickSpec.StepArgumentTransformation
TickSpec.StepAttribute
TickSpec.ParserAttribute
TickSpec.LineSource
TickSpec.ScenarioRun
TickSpec.Table
TickSpec.ScenarioGen
TickSpec.ServiceProvider
TickSpec.FeatureSource
TickSpec.ScenarioSource

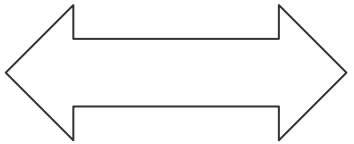


In theory and in practice, unmoderated intra-component cycles are a disaster

Language mechanisms that enforce acyclicity are "good"

There are significant unresolved tensions here in the major industrial languages

Pattern Matching



Abstraction

Patterns are Everywhere

```
let pat = expr
```
Binding

```
fun pat -> expr
```
Function Values

```
let f pat ... pat = expr
```
Function binding

```
match expr with
| pat -> expr
| pat -> expr
| pat -> expr
```
Match expressions

```
try expr
with
| pat -> expr
| pat -> expr
```
Exception Handling

Sequence expressions

```
{ for pat in expr
  for pat in expr
  let pat = expr
  ...
  -> expr }
```

Patterns are Everywhere

```
{ new type with
      member x.M1(pat,...,pat) = expr
      ...
      member x.MN(pat,...,pat) = expr
}
```
Object expressions

```
type C(pat,...,pat) =
    class
        member x.M(pat,...,pat) = expr
        ...
        member x.M(pat,...,pat) = expr
    end
```
Class definitions

Patterns are incredibly useful...

A major part of the enduring appeal of typed functional languages

They are a fundamental programmer's workhorse in F# code

However...

Patterns are incredibly bad...

Traditionally no pattern matching on abstract types

Hence encourage people to break abstraction boundaries

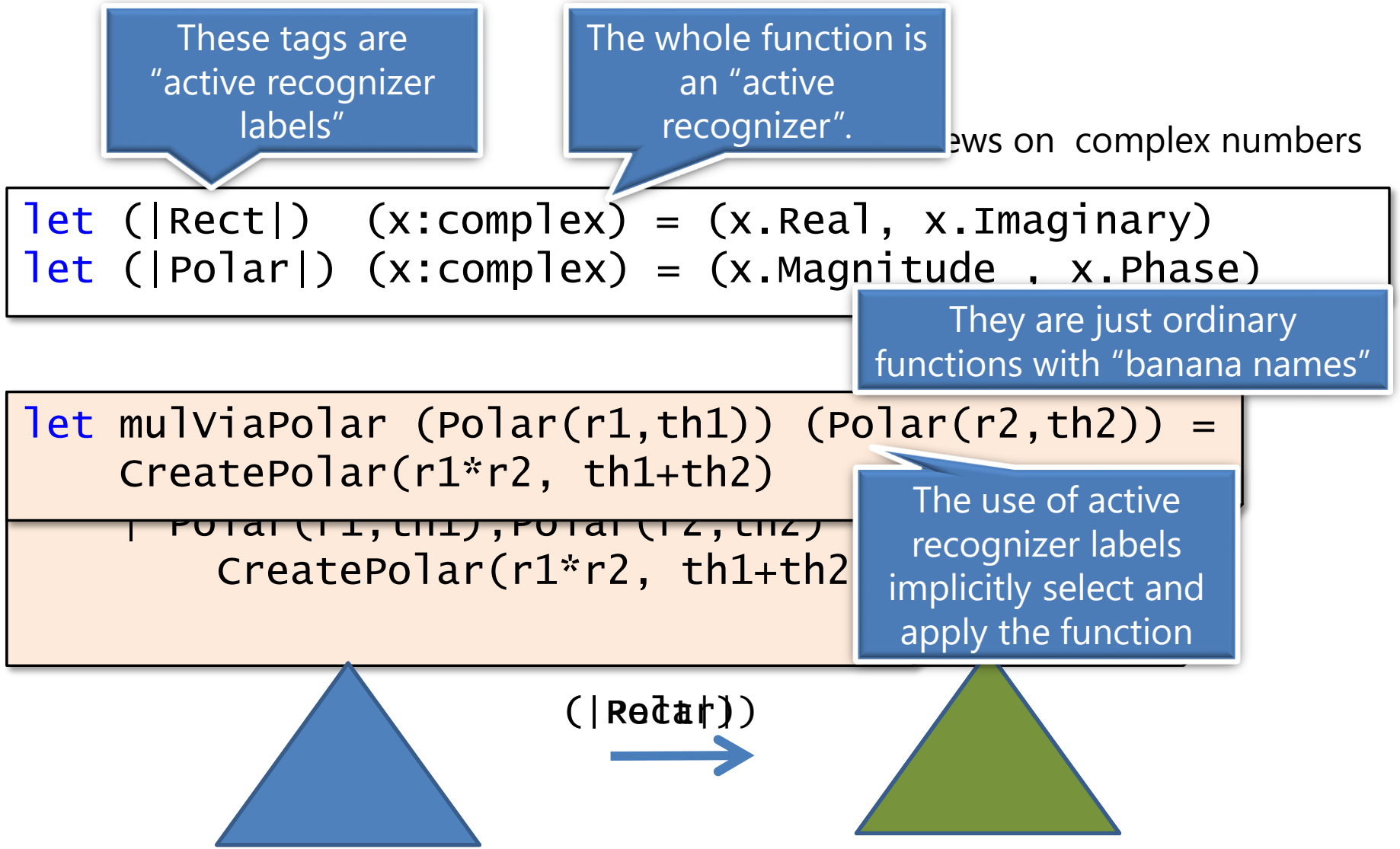
They are not extensible

- Only one view on a "type" is allowed
- Hence mostly effective for sophisticated analysis on term structures in internal implementations

Enter F# Active Patterns….

(Similar to Scala extractors, designed at the same time in loose cooperation)

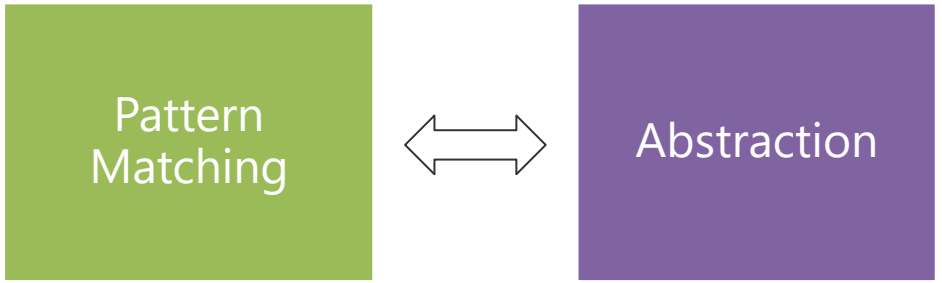
Active Patterns in F#



```fsharp
let (|``Even Number``|``Odd Number``|) n =
    if n % 2 = 0 then ``Even Number``
    else ``Odd Number``

match 312 with
| ``Even Number`` -> "even!"
| ``Odd Number`` -> "odd!"
```

```
module BasicPatterns =

    let (|Value|_|) (e:FSharpExpr) = ...

    let (|Const|_|) (e:FSharpExpr) = ...

    let (|TypeLambda|_|) (e:FSharpExpr) = ...

    let (|Lambda|_|) (e:FSharpExpr) = ...

    let (|Application|_|) (e:FSharpExpr) = ...

    let (|IfThenElse|_|) (e:FSharpExpr) = ...

    let (|Let|_|) (e:FSharpExpr) = ...

    let (|LetRec|_|) (e:FSharpExpr) = ...

    let (|NewRecord|_|) (e:FSharpExpr) = ...

    let (|NewUnionCase|_|) (e:FSharpExpr) = ...

    let (|NewTuple|_|) (e:FSharpExpr) = ...

    let (|TupleGet|_|) (e:FSharpExpr) = ...
```

```
let (|IdentifierNameSyntax|_|) (n: CSharpSyntaxNode) =
    match n with
    | :? IdentifierNameSyntax as t -> Some(t.CSharpKind(), t.Identifier)
    | _ -> None

let (|IdentifierName|_|) (n: CSharpSyntaxNode) =
    match n.CSharpKind() with
    | SyntaxKind.IdentifierName -> ...
    | _ -> None

let (|QualifiedNameSyntax|_|) (n: CSharpSyntaxNode) =
    match n with
    | :? QualifiedNameSyntax as t -> ...
    | _ -> None

let (|QualifiedName|_|) (n: CSharpSyntaxNode) =
    match n.CSharpKind() with
    | SyntaxKind.QualifiedName -> ...
    | _ -> None
```



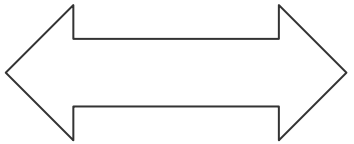
Relaxation Achieved ✓

An extremely useful mechanism that greatly expands the utility of pattern matching in the language

Don’t leave home without them

Code

In one large company, ~500 people work on tooling for this

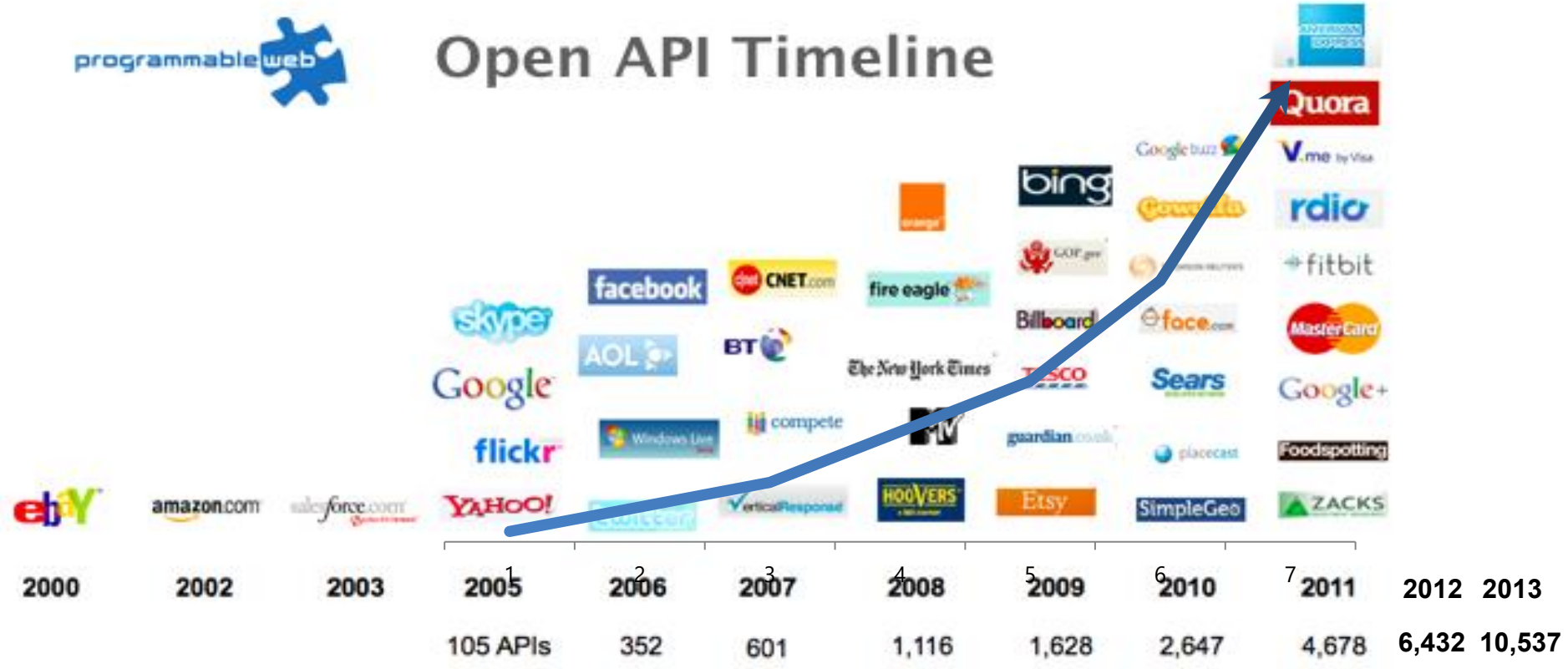


Data

…and 5000+ people work on tooling for this

We are living through an information revolution

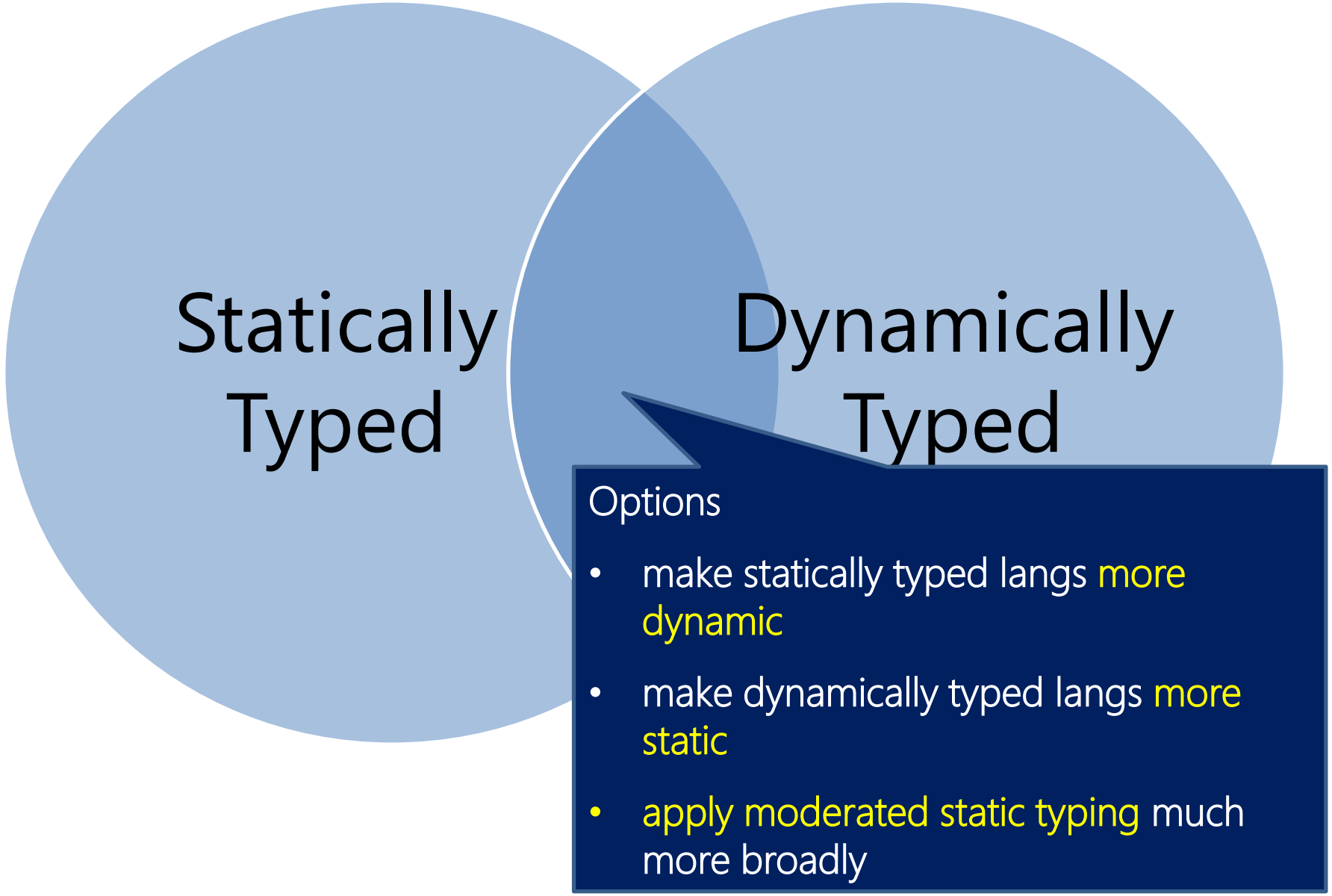
The Information Revolution



We need to bring information **into** the language…

At **internet-scale**, **strongly tooled**, **strongly typed**

Paradigm Locator



http://fsharp.github.io/FSharp.Data/images/csv.gif

http://fsharp.github.io/FSharp.Data/images/wb.gif

Enter F# Type Providers….

"Just like a library"

"A design-time component that computes a space of types and methods on-demand…"

"An adaptor between data/services and the .NET type system…"

"On-demand, scalable compile-time provision of type/module definitions…"

Theme #1 - Many Data Sources, One Mechanism

SQL #1

```
type NorthwndDb =
    SqlDataConnection<ConnectionString = @"AttachDBFileName  = 'C:\project

let db = NorthwndDb.GetDataContext()

let customerNames =
    query { for c in db. do
            where  (c.Ci
            select c.Con
```

AlphabeticalListOfProducts
Categories
CategorySalesFor1997s

property
NorthwndDb.ServiceTypes.Simpl
phabeticalListOfProducts:
System.Data.Linq.Table<Northw

SQL #2

```
let connectionString = @"Data Source=(LocalDb)\v11.0;Initial Catalog=Adventu

[<Literal>]
let query = "
    SELECT TOP(@TopN) FirstName, LastName, SalesYTD
    FROM Sales.vSalesPerson
    WHERE CountryRegionName = @regionName AND SalesYTD > @salesMoreThan
    ORDER BY SalesYTD
"

type SalesPersonQuery = SqlCommandProvider<query, connectionString>
let cmd = SalesPersonQuery()
```

CSV

```
open FSharp.Data

type Stocks = CsvProvider<"C:\\data\\aapl.csv">
let fb = Stocks.Load("http://ichart.finance.yahoo.com/table.csv?s=FB")

for r in fb.Rows do
  if r.
```

- Adj Close
- Close
- Date
- High
- Low
- Open
- Volume

JSON

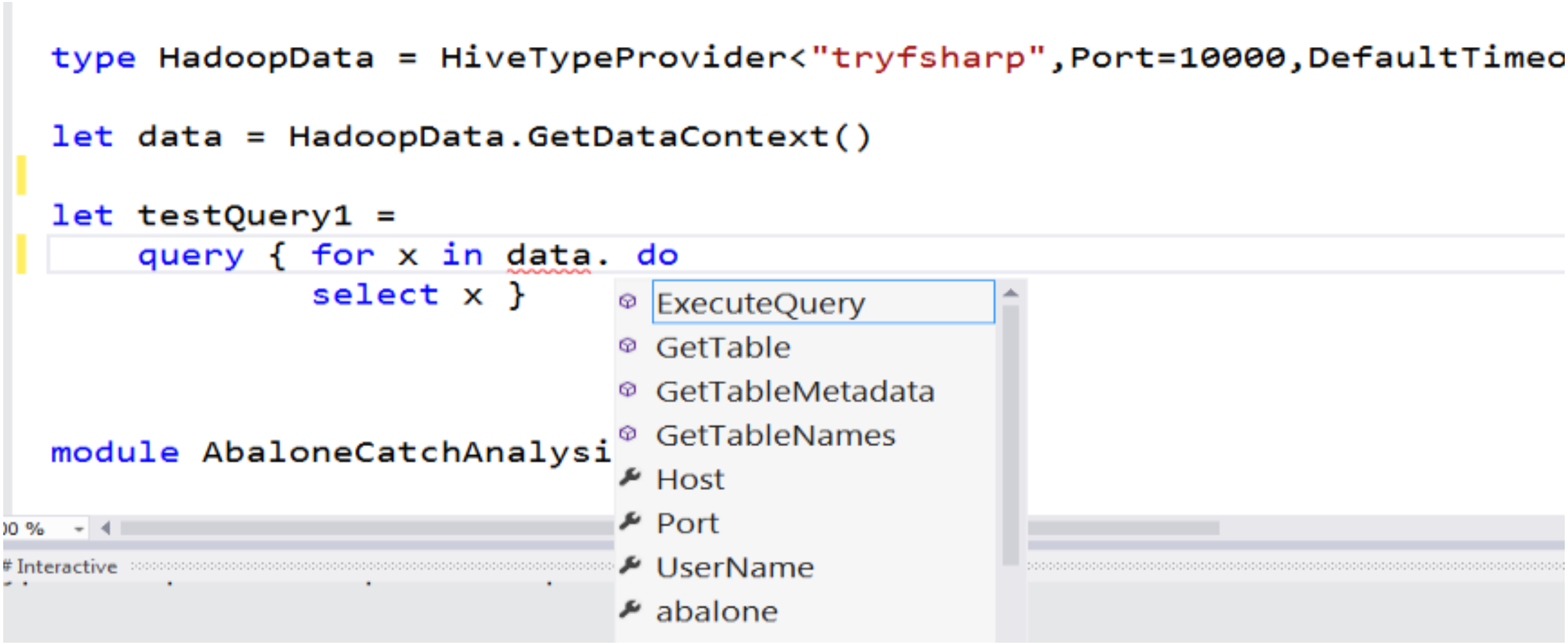
```
open FSharp.Data

[<Literal>]
let sample = "http://api.openweathermap.org/data/2.5/weather?q=London"
let apiUrl = "http://api.openweathermap.org/data/2.5/weather?q="

type Weather = JsonProvider<sample>

let sf = Weather.Load(apiUrl + "San Francisco")
sf.Sys.
```

```
Country        property JsonProvider<...>.Sys.Country: string
JsonValue
Message
Sunrise
Sunset
```

XML

```
1: type Author = XmlProvider<"""<author name="Paul Feyerabend" born="1924" />""">
2: let sample = Author.Parse("""<author name="Karl Popper" born="1902" />""")
3:
4: printfn "%s (%d)" sample.Name sample.Born
```

Hadoop/Hive


type HadoopData = HiveTypeProvider<"tryfsharp",Port=10000,DefaultTimeo
let data = HadoopData.GetDataContext()
let testQuery1 =
query { for x in data. do
select x }
module AbaloneCatchAnalysi
ExecuteQuery
GetTable
GetTableMetadata
GetTableNames
Host
Port
UserName
abalone
Interactive

Web APIs

```
#r "../TypeProviders/Debug/net40/Samples.WorldBank.dll"

let data = Samples.WorldBank.GetDataContext()


data.Countries.

data.Countries.                                    -14 (% of total)
```

- Afghanistan
- Albania
- Algeria
- American Samoa
- Andorra
- Angola
- Antigua and Barbuda
- Arab World

Interactive

Entity Graphs

```
#r @"..\TypeProviders\Debug\net40\Samples.DataStore.Freebase.dll"

open Samples.DataStore.Freebase


// Access the service types using our API key
type Freebase = FreebaseDataProvider<Key=API_KEY>
let ctxt = Freebase.GetDataContext()

ctxt.``Arts and Entertainment``.
```

- Books
- Broadcast
- Comics
- Fictional Universes
- Film
- Games
- Media
- Music

property FreebaseDataProvider<...>.ServiceTypes.Dor
Entertainment.Books:
FreebaseDataProvider<...>.ServiceTypes.Dor
main

The publishing domain is home to most aspe
and the written word -- books, magazines, s
academic papers, etc. Most of the data we h
imported from Wikipedia, although we are l
other possible data sources. We encourage
authors, writings, or publications if we're mis
information, please see the documentation f

%

Interactive

l data : HiveTypeProvider<...>.DataTypes

WSDL

```
type TerraService = WsdlService<"http://msrmaps.com/TerraService2.asmx?WSDL">

let terraClient = TerraService.GetTerraServiceSoap ()
    let myPlace = new TerraService.ServiceTypes.msrmaps.com.Place(City = "Redı
    let myLocation = terraClient.ConvertPlaceToLonLatPt(myPlace)
    printfn "Redmond Latitude: %f Longitude: %f" (myLocation.Lat) (myLocation
```

R

```
// Pull in stock prices for some tickers then compute returns
let data = [
    for ticker in [ "MSFT"; "AAPL"; "VXX"; "SPX"; "GLD" ] ->
        ticker, getStockPrices ticker 255 |> R.log |> R.diff ]

// Construct an R data.frame then plot pairs of returns
let df = R.data_frame(namedParams data)
R.pairs(df)
```

Theme #2 - Data at Multiple Scales

From Everything to Individuals

```
data.AllEntites
```

Data Scripters need to work with different granularities of schematization

```
data.Automotive.``Automobile Models``
```

```
data.Automotive.``Automobile Models``.Individuals.``Porsche 911``
```

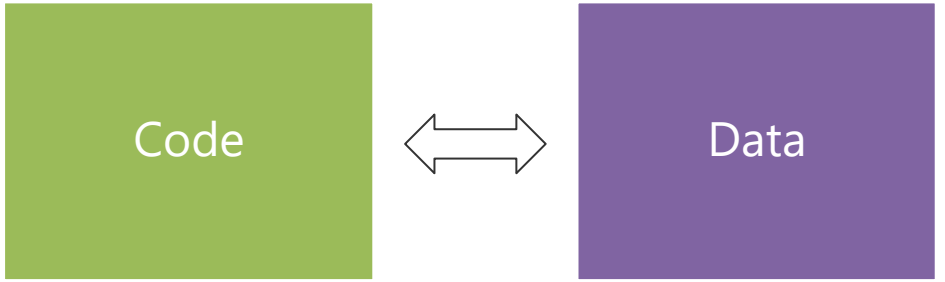
…Only a language that supports massively scalable metadata can operate at all these levels

Every stable entity can get a unique type

"The idea of integrating internet-scale information services directly into the program's variable and type space is probably the most innovative programming language idea I've heard of in a decade"

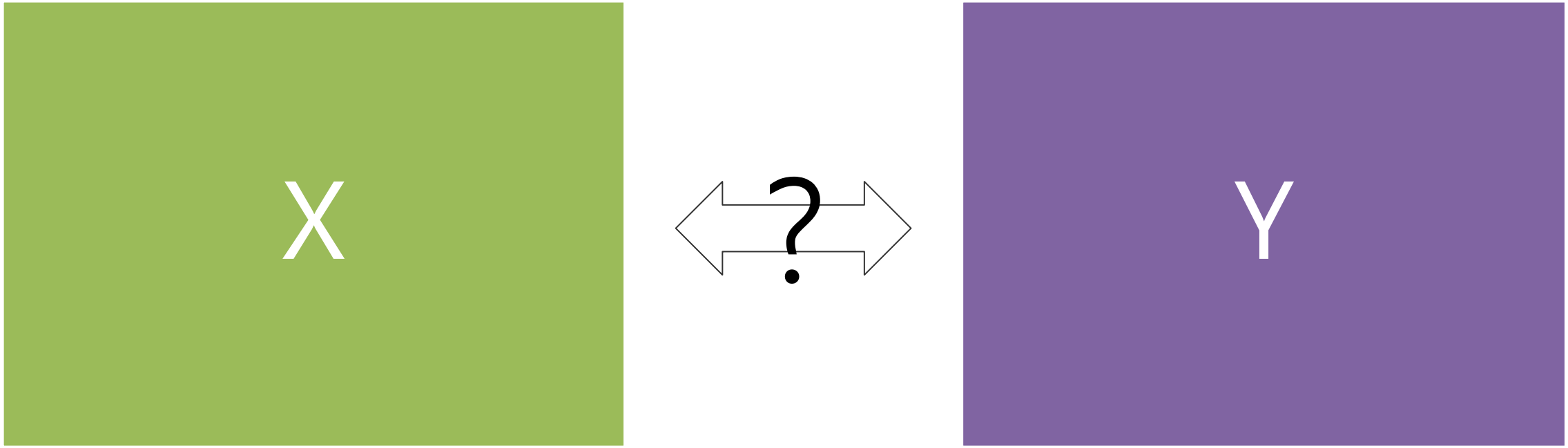
(Jim Cordy, Queens U)

Overly kind, but thought provoking!

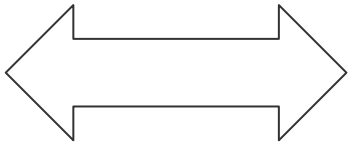


Relaxation Achieved ✓ ✓

F# Type Providers have been a hugely successful feature – their range of useful application is enormous. They bring data and typed programming into synthesis like no other feature around

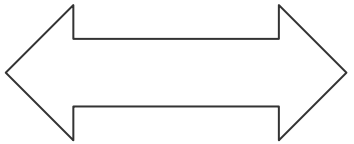

X
?
Y

Sync



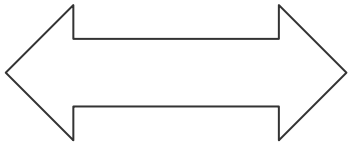
Async

Numbers



Numbers-with Units

GPU



CPU

http://fsharp.org/use/gpu ✓

Alea GPU

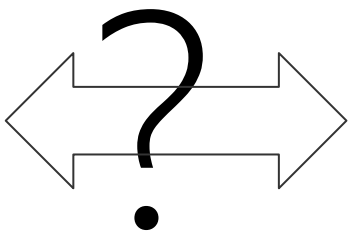
FSCL

What's Ahead?

Well, for me, many areas of tension in "upstack" language design have been resolved

But there are still some areas that bug me…

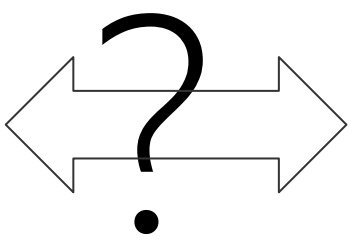
REPL



Distribution and Scale

http://m-brace.net

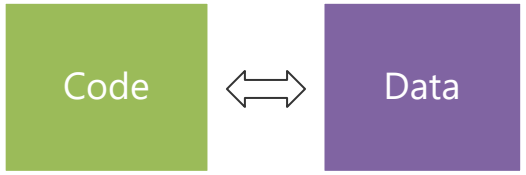
Rows

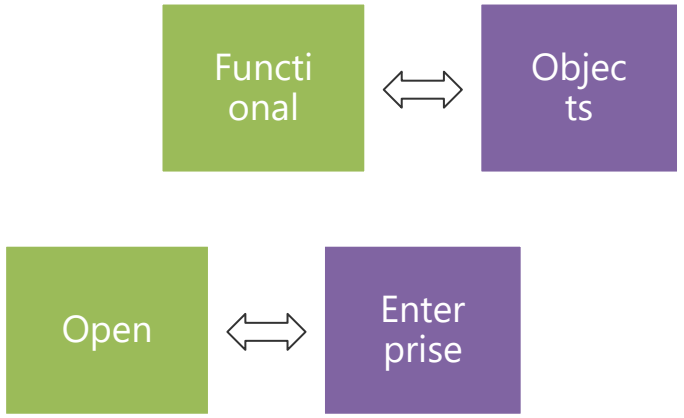


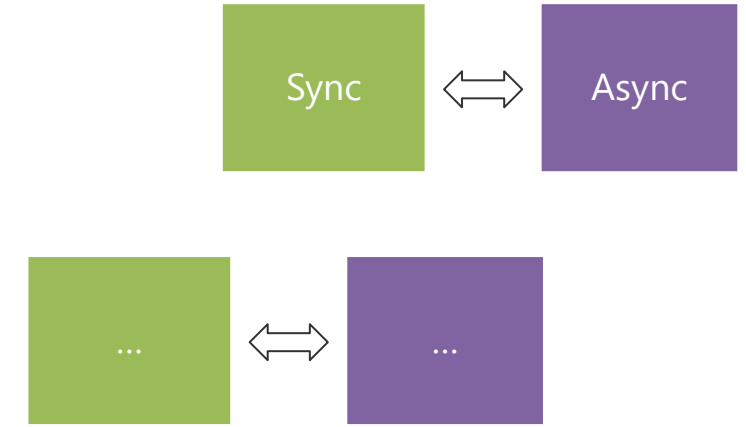
Columns

e.g. Trill - VLDB ‘15

In Conclusion!







Synthesis from contradiction is at the heart of applied PL design

Opposites come in all shapes and flavours

Achieving simplicity and relaxation is key ☺

Thank you!!

Questions?